

PrepMaster
A PROJECT REPORT

Submitted by

Sneh Ghetiya – 924008089
Dev Kalola – 92400938125
Daksh Hadvani – 92400938128
Kashyap Ladani – 92400938142
Harsh Joshi - 92400938152

In partial fulfilment for the award of the degree of

DIPLOMA ENGINEERING

In

Computer Engineering



Marwadi
University
Marwadi Chandarana Group





Marwadi
University
Marwadi Chandarana Group



Marwadi University, Rajkot

Faculty of Diploma Studies

Computer Engineering Department

2026-27

CERTIFICATE

This is to certify that the project entitled **PrepMaster** has been carried out by **Sneh Ghetiya (92400938089)** under my guidance in partial fulfilment of the degree of Diploma Engineering in Computer Engineering (5th Semester) of Marwadi University, Rajkot during the academic year 2026-27.

Date : 08-08-2026

Internal Guide

Prof. Sumit Makwana

Assistant Professor

Head of the Department

Prof. Smit Thacker

Computer Engineering



Marwadi
University
Marwadi Chandarana Group



Marwadi University, Rajkot

Faculty of Diploma Studies

Computer Engineering Department

2026-27

CERTIFICATE

This is to certify that the project entitled **PrepMaster** has been carried out by **Dev Kalola (92400938125)** under my guidance in partial fulfilment of the degree of Diploma Engineering in Computer Engineering (5th Semester) of Marwadi University, Rajkot during the academic year 2026-27.

Date : 08-08-2026

Internal Guide

Prof. Sumit Makwana

Assistant Professor

Head of the Department

Prof. Smit Thacker

Computer Engineering



Marwadi
University
Marwadi Chandarana Group



Marwadi University, Rajkot

Faculty of Diploma Studies

Computer Engineering Department

2026-27

CERTIFICATE

This is to certify that the project entitled **PrepMaster** has been carried out by **Daksh Hadvani (92400938128)** under my guidance in partial fulfilment of the degree of Diploma Engineering in Computer Engineering (5th Semester) of Marwadi University, Rajkot during the academic year 2026-27.

Date : 08-08-2026

Internal Guide

Prof. Sumit Makwana

Assistant Professor

Head of the Department

Prof. Smit Thacker

Computer Engineering



Marwadi
University
Marwadi Chandarana Group



Marwadi University, Rajkot

Faculty of Diploma Studies

Computer Engineering Department

2026-27

CERTIFICATE

This is to certify that the project entitled **PrepMaster** has been carried out by **Kashyap Ladani (92400938142)** under my guidance in partial fulfilment of the degree of Diploma Engineering in Computer Engineering (5th Semester) of Marwadi University, Rajkot during the academic year 2026-27.

Date : 08-08-2026

Internal Guide

Prof. Sumit Makwana

Assistant Professor

Head of the Department

Prof. Smit Thacker

Computer Engineering



Marwadi
University
Marwadi Chandarana Group



Marwadi University, Rajkot

Faculty of Diploma Studies

Computer Engineering Department

2026-27

CERTIFICATE

This is to certify that the project entitled **PrepMaster** has been carried out by **Harsh Joshi (92400938152)** under my guidance in partial fulfilment of the degree of Diploma Engineering in Computer Engineering (5th Semester) of Marwadi University, Rajkot during the academic year 2026-27.

Date : 08-08-2026

Internal Guide

Prof. Sumit Makwana

Assistant Professor

Head of the Department

Prof. Smit Thacker

Computer Engineering

Contents

Acknowledgements	I
Abstract	II
List of Tables	III
List of Figures	IV
Diagrams	V
1. Introduction	13
1.1 Document purpose	14
1.2 Product scope	14
1.3 Intended audience and document overview	14
1.4 Definitions and abbreviations	14
1.5 Document conventions	15
1.6 References and acknowledgments	16
2 Overall description	17
2.1 Product perspective	17
2.2 Product functionality	17
2.2 Users and characteristics	18
2.4 Operating environment	19
2.5 Design and implementation constraints	19
2.6 User documentation	20
2.7 Assumptions and dependencies	20
3 Specific requirements	22
3.1 External interface requirements	22
3.2 Functional requirements	25
3.3 Behavioral requirements	26
4 Other non-functional requirements	27
4.1 Performance requirements	27
4.2 Safety and security requirements	27
4.3 Software quality attributes	28
Appendix A – Data dictionary	

Acknowledgement

The presentation of this report brings us immense satisfaction and a sense of achievement as we present this document on our project work as a final fulfillment of our Diploma in Computer Engineering journey. This report stands as a testament to our dedication, hard work, and commitment.

We want to take this chance to express our deep gratitude to all those who provided unwavering support and encouragement throughout this project. Their contributions have played a vital role in making this project a success. Without them, this project would have been much more challenging to accomplish.

First and foremost, we extend our deep appreciation to Prof. Sumit Makwana for her invaluable guidance and support throughout the course of our project. Her insights and mentorship played a pivotal role in helping us achieve the desired outcomes. Equally deserving of our gratitude are all the faculty members who have played a crucial role in our academic journey.

We acknowledge with immense gratitude the pivotal role played by our project guide and other faculty members. Their consistent guidance and motivation have not only pointed us in the right direction but have also instilled in us a sense of confidence to excel in our project.

Our gratitude extends to all who have contributed to shaping our project and enriching our understanding.

We are profoundly thankful to all those who have supported us, offering their wisdom, guidance, and encouragement. Your contributions have been instrumental in the success of this project, and we are truly grateful for your unwavering support.

Abstract

Most university students struggle during exam time not because they don't study, but because they don't have the right material to study from. They rely on random YouTube videos, scattered notes, or generic quiz apps that have no connection to their actual university syllabus. There's no single tool that understands what semester they're in, what subjects they're studying, or what their university specifically expects from them in the exam.

PrepMaster is a university exam preparation platform built to fix exactly that. It comes preloaded with around more than 400+ curated questions for each exam, organized semester-by-semester and subject-by-subject, so every question a student practices is directly relevant to their actual coursework. The platform is built using React JS for a fast and responsive interface, styled with Tailwind CSS, and enhanced with Framer Motion for smooth and engaging animations — all backed by Firebase for real-time data storage and seamless question delivery. The AI layer on top provides context-aware guidance that maps directly to the university's syllabus, so nothing ever feels out of place.

What makes PrepMaster genuinely different from any existing study tool is its specificity. It isn't trying to serve every student everywhere — it's built ground-up around one university's curriculum, its subjects, its semesters, and its exam patterns. Students don't waste time filtering out irrelevant content or wondering if what they're studying will actually appear in their paper. Everything is already structured for them, making exam preparation smarter, faster, and a lot less stressful .

Functionalities :

1.Admin Panel

In this panel functionalities are Admin dashboard , Add / Remove admin , Mark student as spam, Edit course / subjects / questions , Review / flag feedbacks , Check leaderboards .

2. Student Panel

In this panel functionalities are Student dashboard , Give tests , Create / Join testrooms , Give feedbacks , Review analysis , Change question preference

List of Table

1	Table 1.1 Students	30
2	Table 1.2 Admins	30
3	Table 1.3 TestRooms	30
4	Table 1.4 TestAttempts	31
5	Table 1.5 Subjects	31
6	Table 1.6 Courses	31
7	Table 1.7 TestsData	31
8	Table 1.8 Feedbacks	32
9	Table 1.9 Leaderboards	32
10	Table 1.10 Subscribers	32
11	Table 1.11 Subscriptions	33

List of Figures

1	Figure 1.1 Usecase Diagram	1
2	Figure 2.1 Student Flowchart Diagram	2
3	Figure 2.2 Admin Flowchart Diagram	3
4	Figure 3.1 Student Activity Diagram	4
5	Figure 3.2 Admin Activity Diagram	5
6	Figure 4.1 Student Sequence Diagram	6
7	Figure 4.2 Admin Sequence Diagram	7
8	Figure 5.1 E.R Diagram	8
9	Figure 6.1 Data Flow Diagram (Level 0)	9
10	Figure 6.2 Data Flow Diagram (Level 1)	10
11	Figure 6.3 Data Flow Diagram (Level 2)	11
12	Figure 7.1 Class Diagram	12

Diagrams

Usecase Diagram :

PrepMaster Usecase Diagram



Figure 1.1 Usecase Diagram

Student Flowchart Diagram :

PREP MASTER - STUDENT FLOW

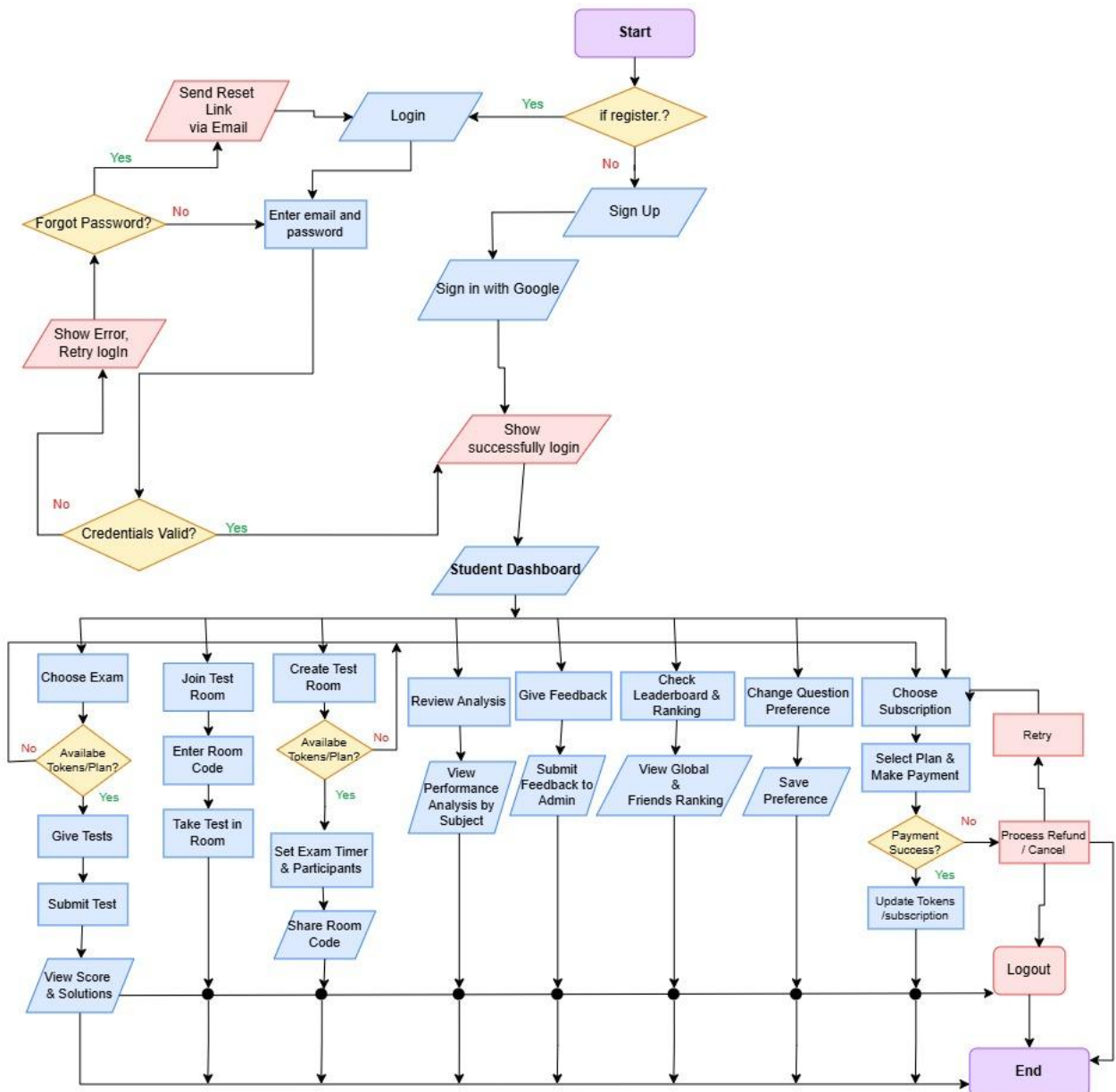


Figure 2.1 Student Flowchart Diagram

Admin Flowchart Diagram :

PREP MASTER - ADMIN FLOW

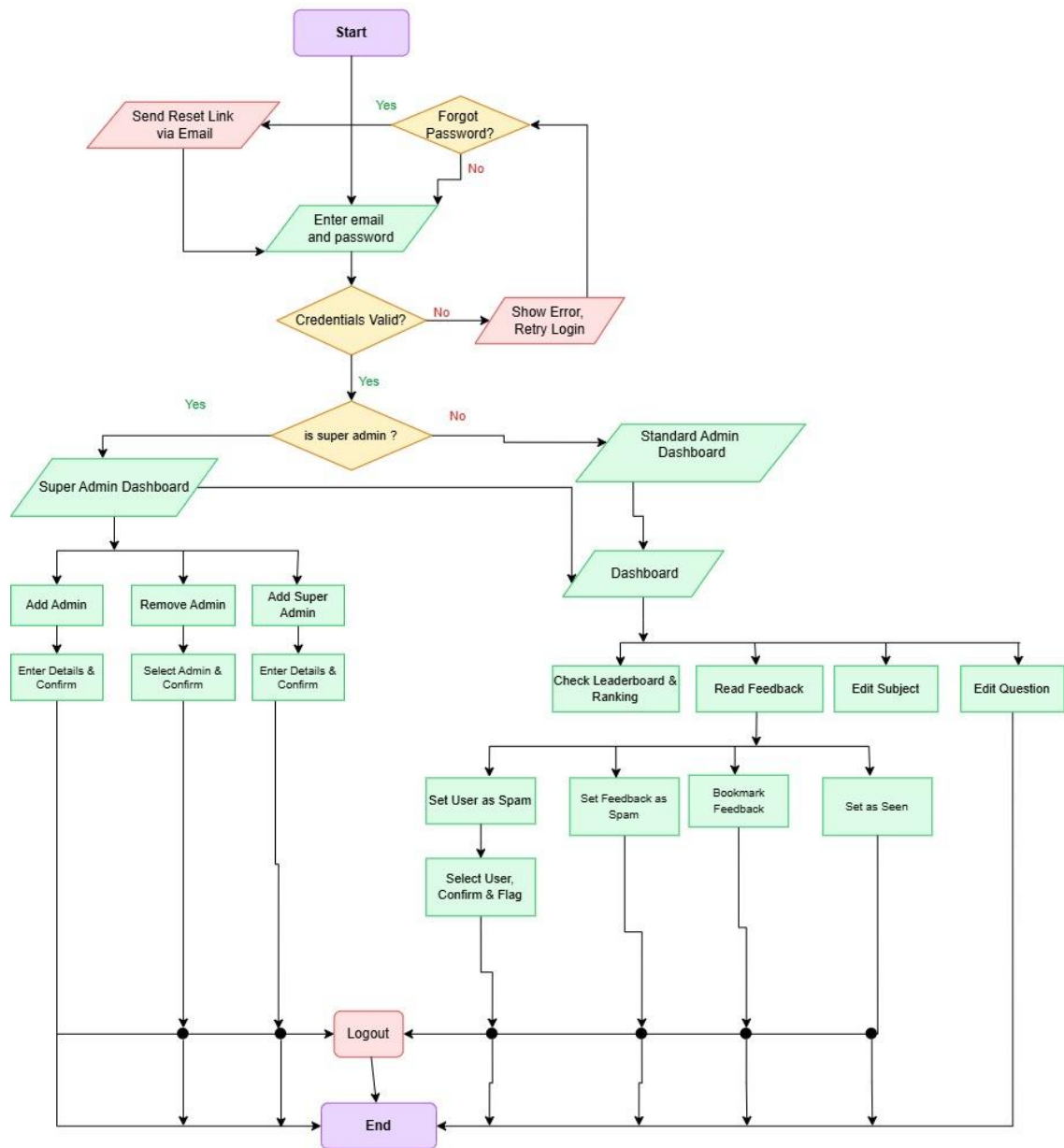


Figure 2.2 Admin Flowchart Diagram

Student Activity Diagram :

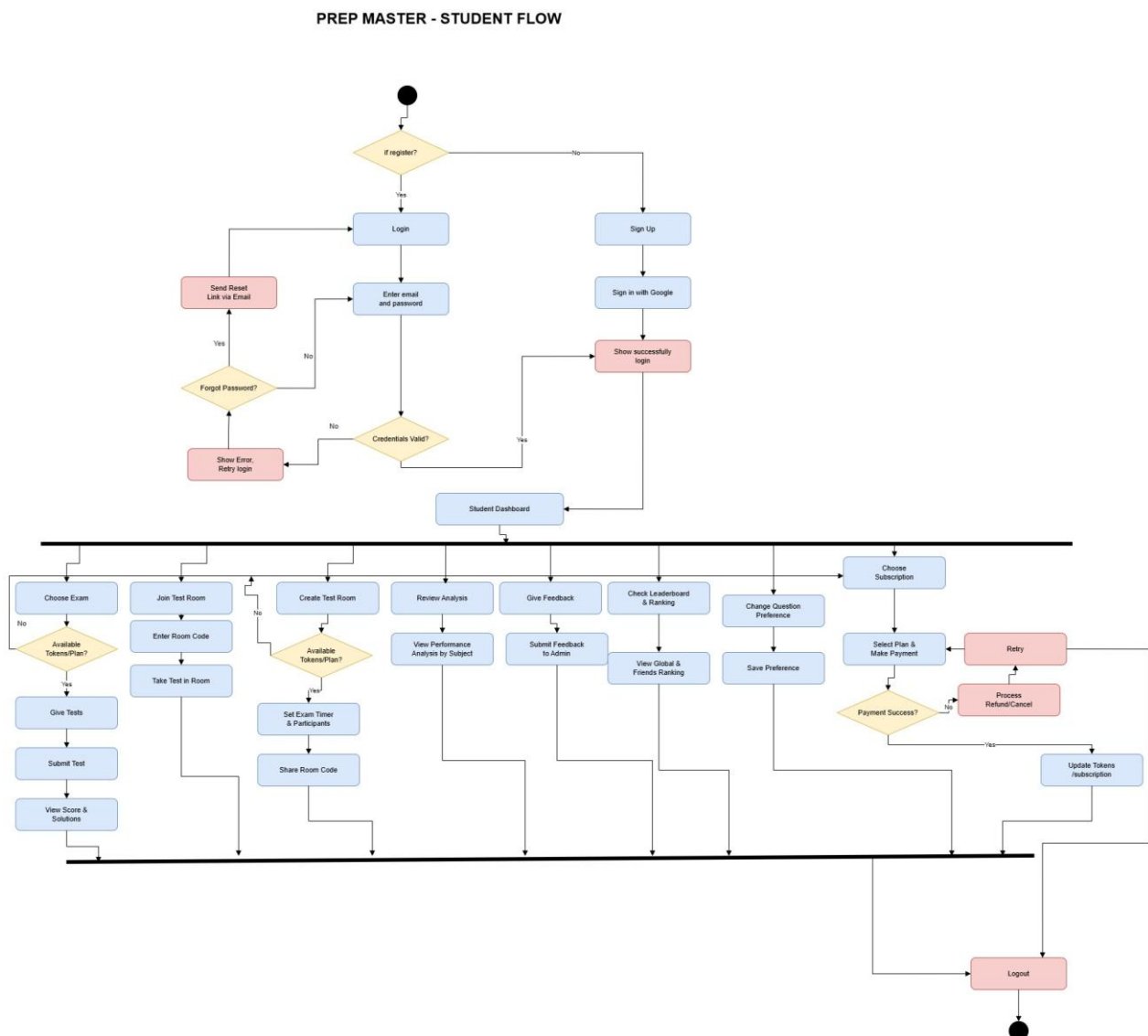


Figure 3.1 Student Activity Diagram

Admin Activity Diagram :

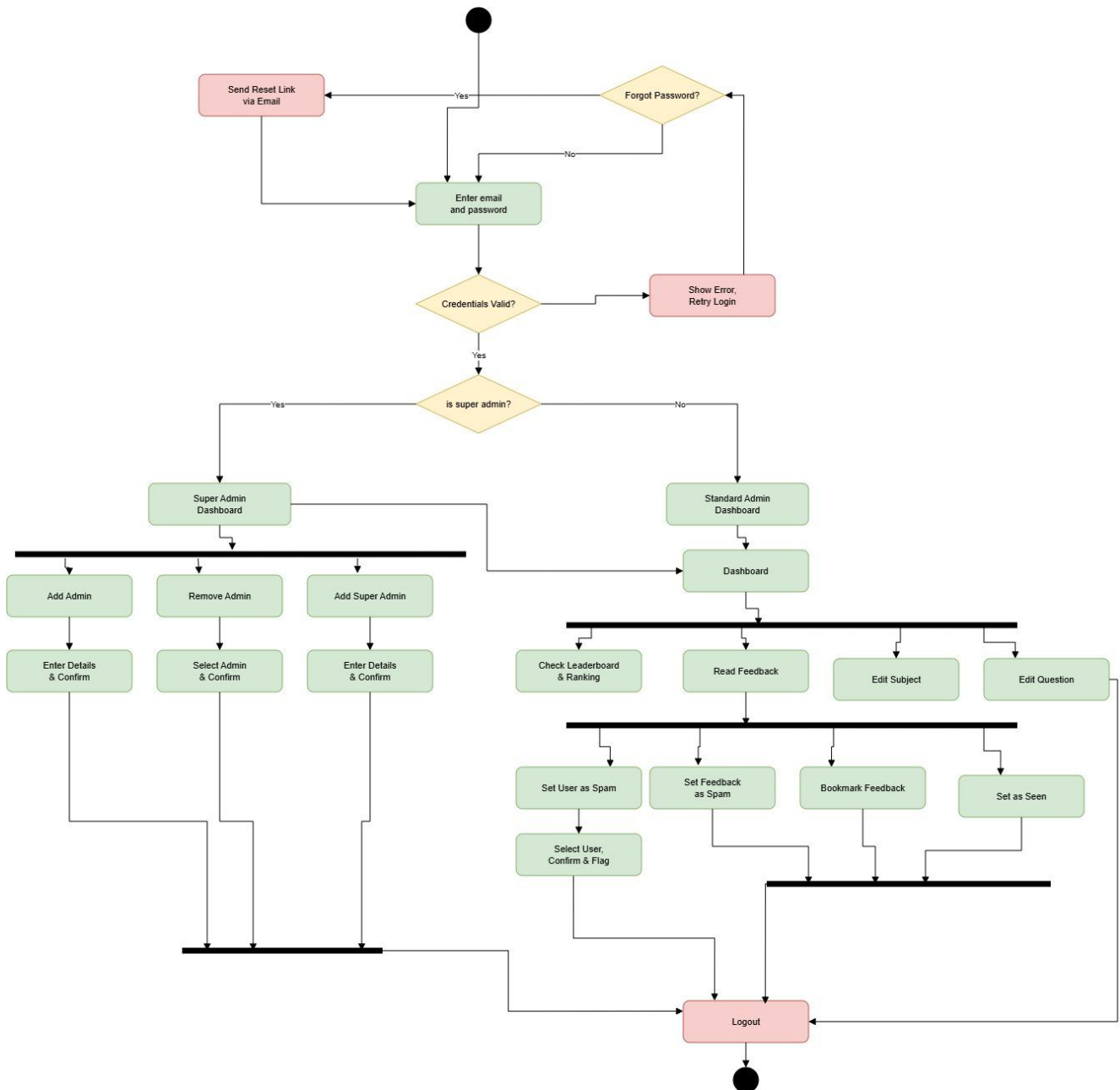


Figure 3.2 Admin Activity Diagram

Student Sequence Diagram :

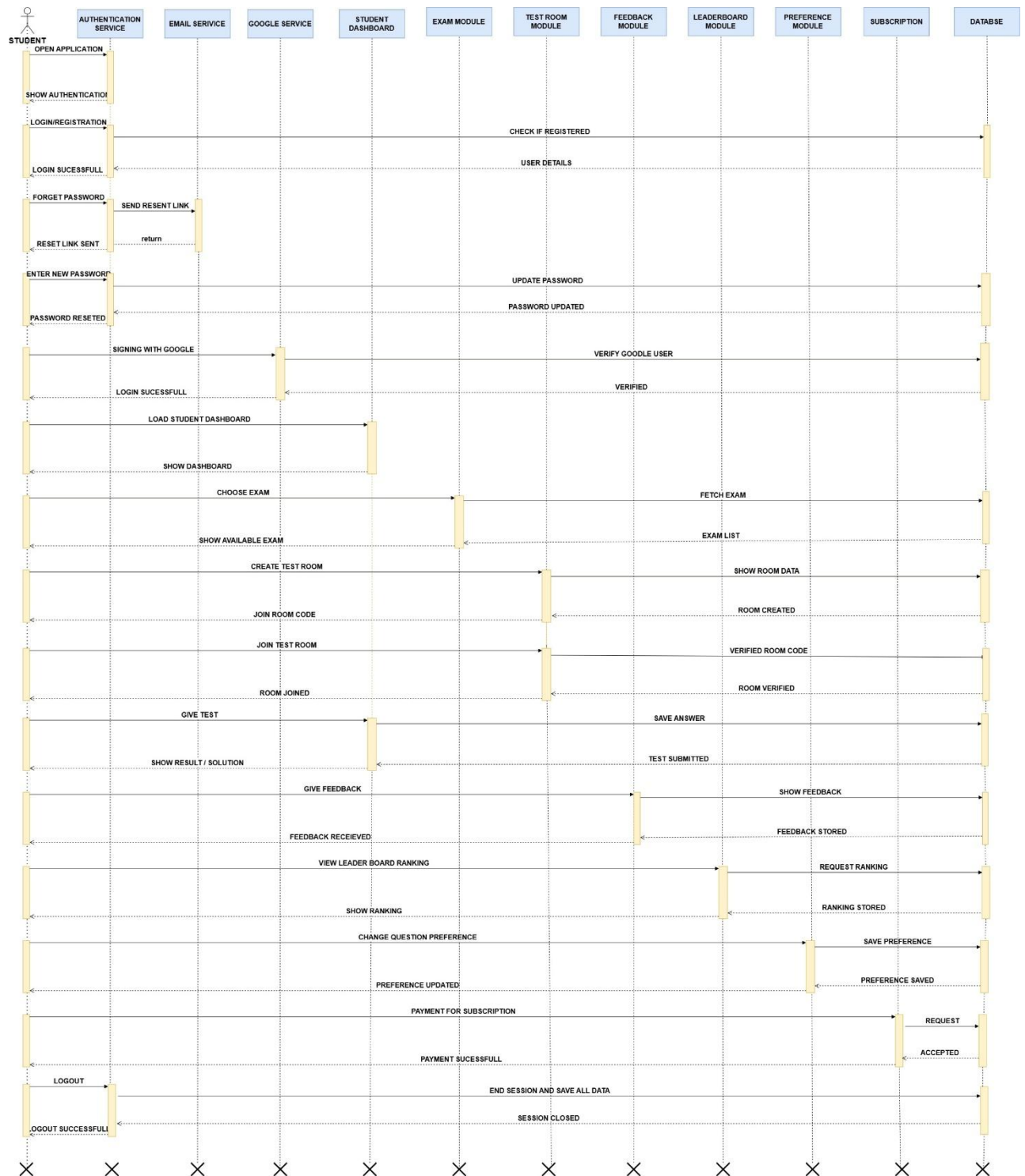


Figure 4.1 Student Sequence Diagram

Admin Sequence Diagram :

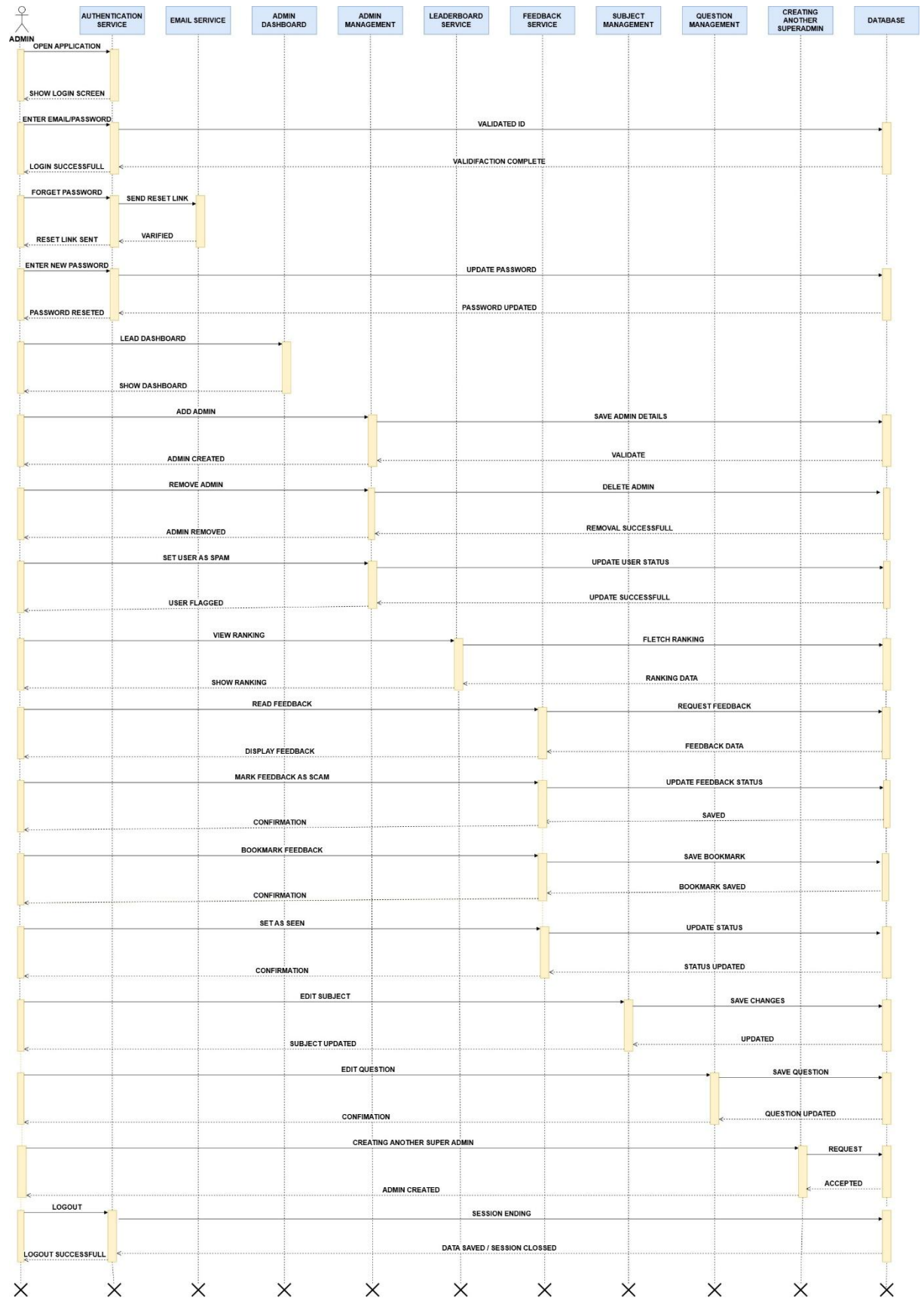


Figure 4.2 Admin Sequence Diagram

E.R Diagram :

PrepMaster ER Diagram

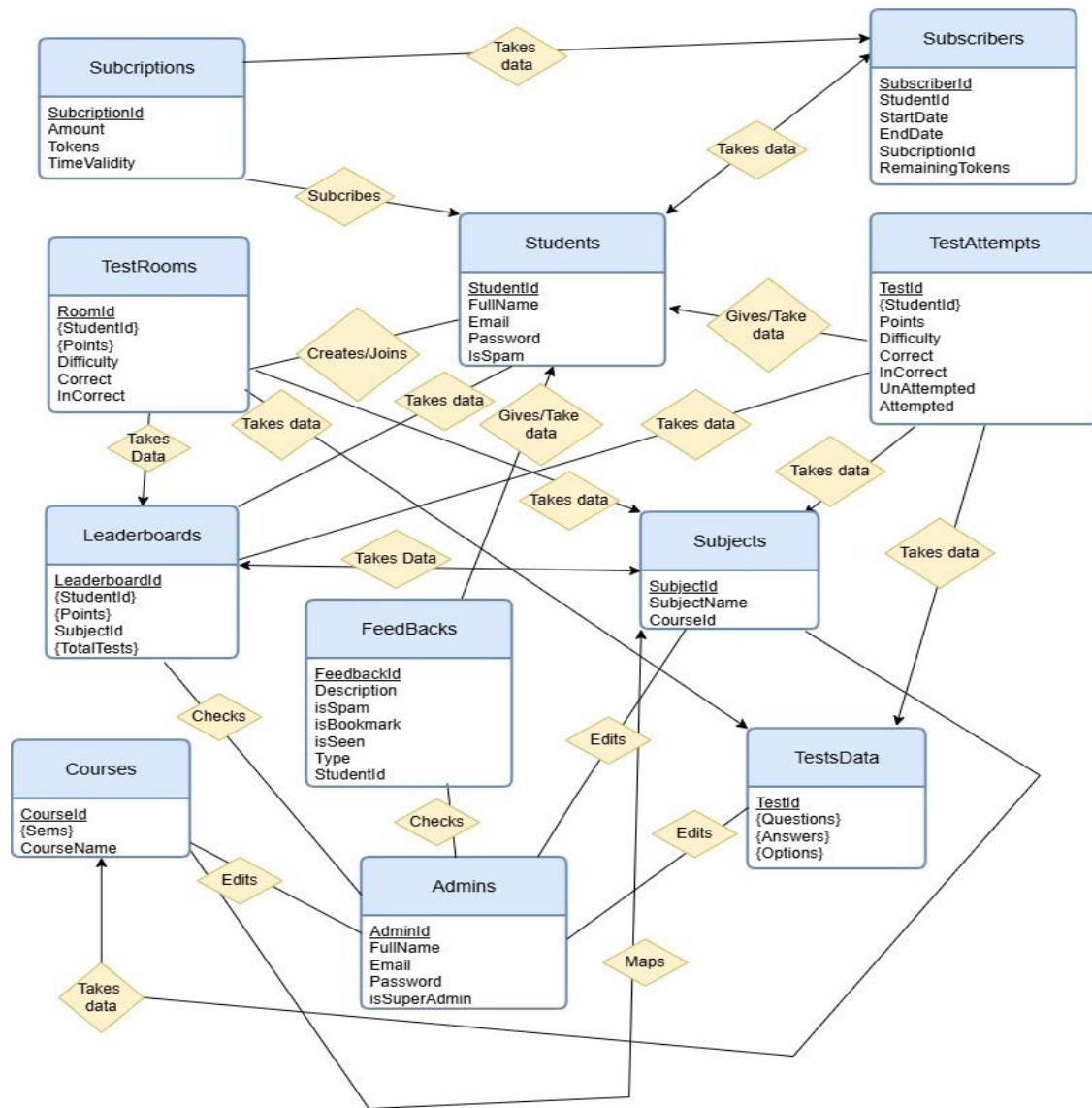


Figure 5.1 E.R Diagram

Data Flow Diagram (Level 0) :

PrepMaster DFD Level 0

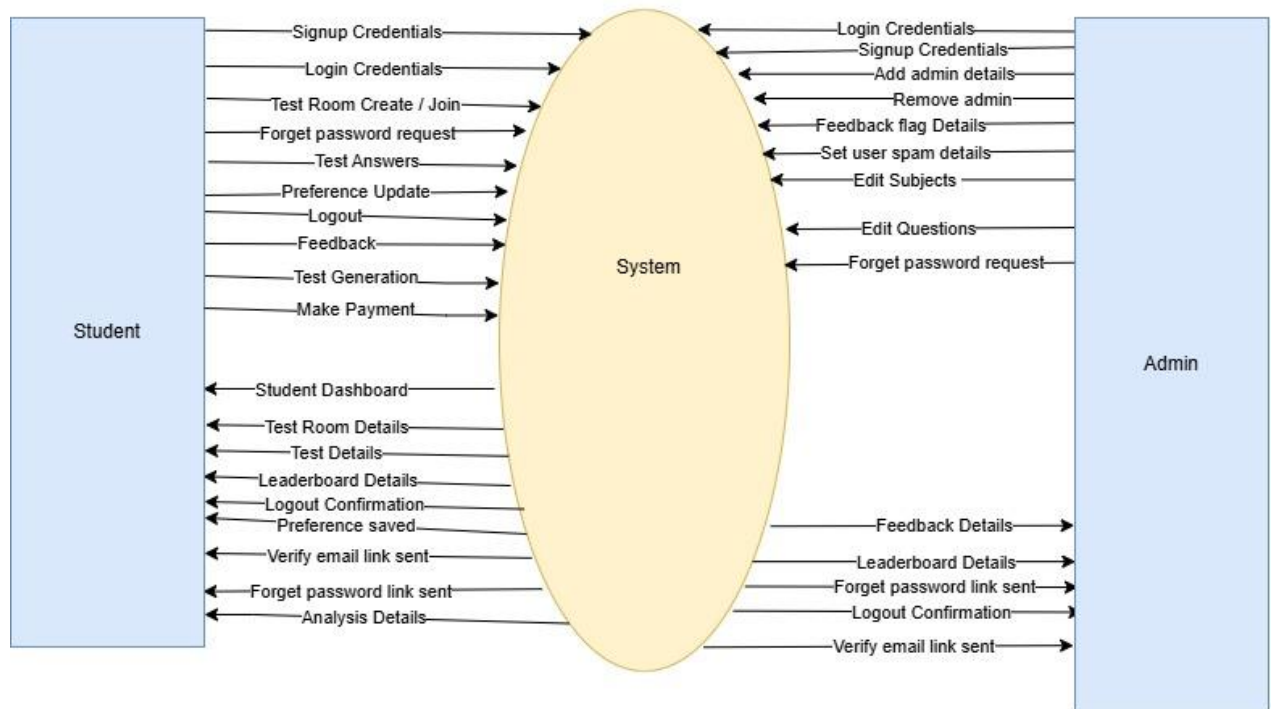


Figure 6.1 Data Flow Diagram (Level0)

Data Flow Diagram (Level 1) :

PrepMaster DFD Level 1

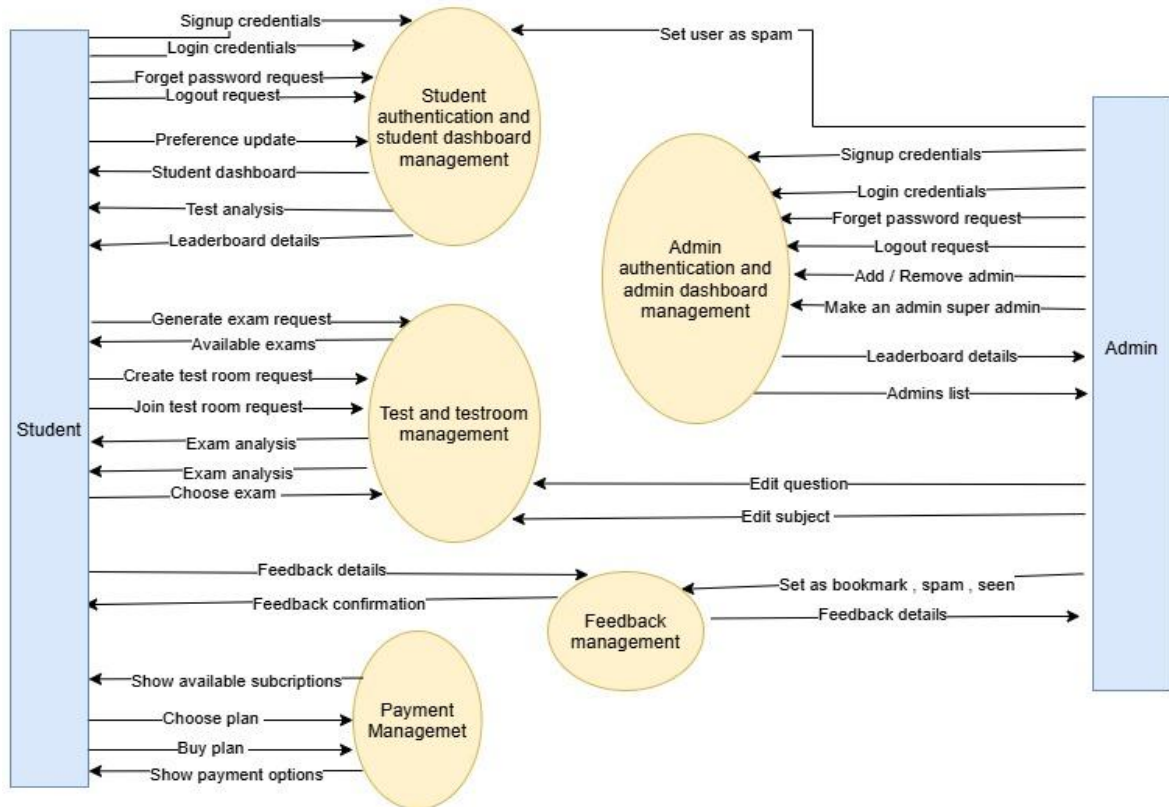


Figure 6.2 Data Flow Diagram (Level 1)

Data Flow Diagram (Level 2) :

PrepMaster DFD Level 2

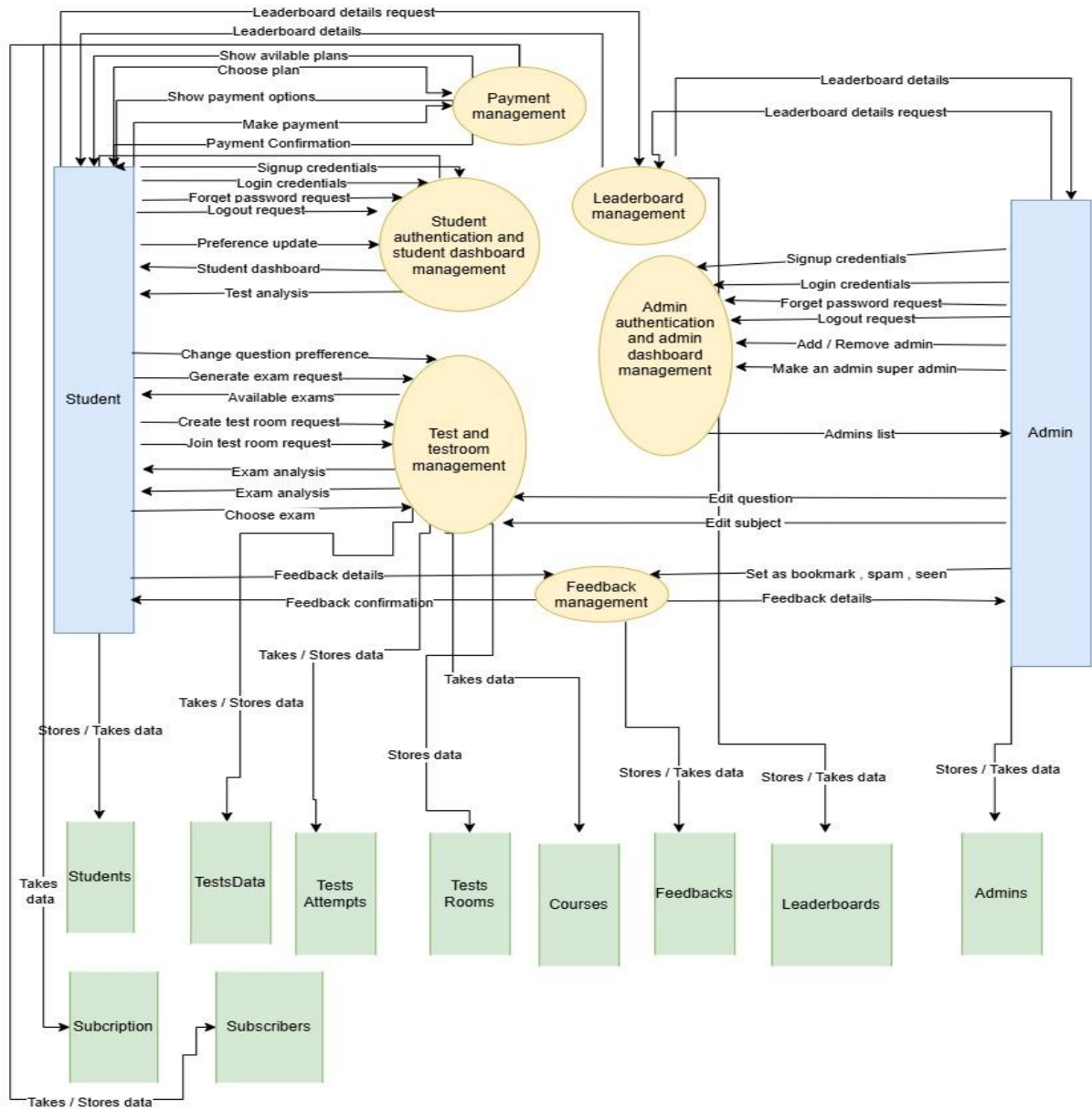


Figure 6.3 Data Flow Diagram (Level 2)

Class Diagram :

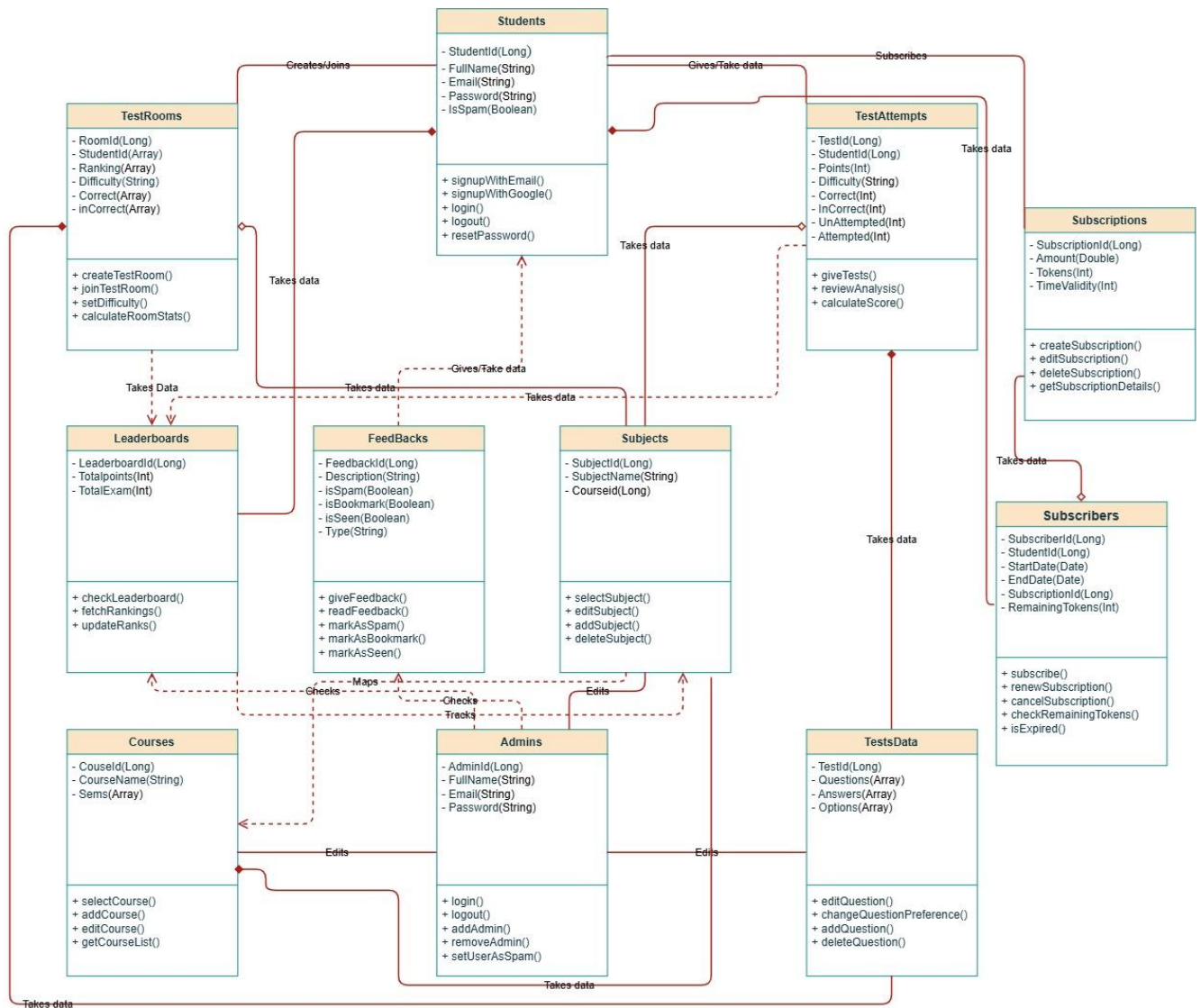


Figure 7.1 Class Diagram

1. Introduction

PrepMaster is a next-generation, AI-driven exam preparation and dynamic performance analytics platform engineered to bridge the gap between static self-study and real-time collaborative learning. In modern academic and competitive exam environments, traditional preparation methods often lack immediate diagnostic feedback, personal adaptive learning tracks, and synchronized peer engagement. PrepMaster addresses these challenges by combining dynamic multi-tenant exam engines, low-latency microservices, and automated performance tracking within a unified web-based application.

The platform provides a dual-interface architecture designed specifically for two distinct primary actors: Students and System Administrators. For students, PrepMaster serves as an interactive testing environment capable of running individual practice tests, generating instant automated scoring solutions, and hosting real-time multiplayer test rooms. By leveraging adaptive question selection algorithms and subject-level performance metrics, students can easily isolate their academic weaknesses, adjust custom difficulty preferences, and compete on global or friend-group leaderboards.

For administrators, PrepMaster acts as a central governance and content moderation hub. The administrative interface provides complete oversight over user account status, automated spam detection, and user access flags. Furthermore, it equips admins with CRUD (Create, Read, Update, Delete) controls over the core content hierarchy—including courses, academic subjects, and underlying test question datasets—while providing structured moderation tools to manage and resolve user feedback tickets.

Architecturally, the application is built on a scalable cloud infrastructure using a modern fullstack web paradigm backed by Firebase Firestore for real-time data persistence and Google OAuth/JWT protocols for secure identity management. High-speed external inference engines are integrated directly into the testing pipelines to handle dynamic question parsing and automated answer evaluation. By combining real-time socket communication, modular database schemas, and clean user experience design, PrepMaster delivers a reliable, secure, and scalable digital assessment platform.

1.1 Document Purpose

This Software Requirements Specification (SRS) document defines the architectural, functional, and non-functional requirements for the PrepMaster web application. It serves as the authoritative reference for software engineers, database architects, UI/UX designers, and project evaluators during development, testing, and deployment.

1.2 Product Scope

PrepMaster is an intelligent, full-stack exam preparation and performance analytics platform. The application empowers students to take practice exams, create and join real-time collaborative test rooms, evaluate subject-wise performance analytics, and compete on global and peer leaderboards.

Additionally, the system equips administrators with central management capabilities to moderate users, flag spam, curate subject catalogues, manage test question data, and resolve platform feedback.

1.3 Intended Audience and Document Overview

This document is designed for project leads, backend and frontend developers, QA engineers, and academic stakeholders.

1.4 Definitions, and Abbreviations

TERMS	DEFINATION
PrepMaster	The web application platform being specified in this document.
SRS	Software Requirements Specification; the official technical requirements document.
Groq API	High-speed AI inference engine utilized for question processing, generation, and keyword detection.

SRS	Software Requirements Specification, the current document
OAuth	Open Authorization protocol used for secure third-party login (e.g., Sign in with Google).
REST API	Representational State Transfer Application Programming Interface for client-server communication.
JWT	JSON Web Token, used for stateless user session authentication.
MERN	Software stack consisting of MongoDB, Express.js, React, and Node.js.
DFD	Data Flow Diagram, representing data movement through the system across various levels.
SDK	Software Development Kit
ERD	Entity-Relationship Diagram, defining structural database entities, attributes, and relationships.
Linux	Lovable Intellect Not Using XP
Vercel	Cloud platform used for hosting and automated deployment of the frontend and serverless routes.
COM Port	Communication Port(Serial Port)
SMTP	Simple Mail Transfer Protocol, used for automated email transactions (e.g., password reset links).

UI/UX	User Interface / User Experience.
-------	-----------------------------------

Table 1.4.1 Terms and Abbreviations

1.5 Document Conventions

- The entire document is formatted according to IEEE (Institute of Electrical and Electronics Engineers) standards.
- The font used for the text is Times New Roman, with a recommended font size of either 11 or 12
- Italics are used to highlight comments, emphasizing additional information or notes.
- The text in the document is single-spaced, and the margins are set to 1 inch, as specified in the provided template.
- Section and subsection titles should adhere to the template's formatting for consistency.

The section acknowledges the potential need to further detail specific standards or conventions, which might involve separate subsections for Formatting Conventions, Naming Conventions, and other relevant categories.

Overall, this section ensures that the document maintains a uniform and professional appearance while adhering to the established IEEE guidelines for formatting and presentation.

1.6 References and Acknowledgments

- **Marwadi University Official Departmental Curriculum & Syllabus Portals:**
- Faculty of Technology (B.Tech Engineering):
<https://www.marwadiuniversity.ac.in/b-tech-engineering-colleges-in-gujarat/>
- Faculty of Business Management (BBA):
<http://marwadiuniversity.ac.in/business-management-bba/>
- Faculty of Computer Applications (BCA):
<https://www.marwadiuniversity.ac.in/bca-main-course/>

- Faculty of Law (LL.B):
<https://www.marwadiuniversity.ac.in/bachelor-of-law-llb-colleges-fees/>
- Faculty of Physiotherapy (BPT):
<https://www.marwadiuniversity.ac.in/bachelor-of-physiotherapy-bpt-colleges-fees/>
- Faculty of Pharmacy (B.Pharm):
<https://www.marwadiuniversity.ac.in/b-pharmacy-colleges-fees/>
- Faculty of Science (B.Sc):
<https://www.marwadiuniversity.ac.in/bachelor-of-science-colleges/>
- Faculty of Technology (Diploma in Engineering):
<https://www.marwadiuniversity.ac.in/diploma-in-engineering/>

2. Overall Description

2.1 Product Perspective

PrepMaster is a web-based, AI-driven exam preparation and performance analytics ecosystem deployed on cloud infrastructure (Vercel edge hosting with serverless API integration). Operating within the modern competitive exam and higher-education domain, PrepMaster bridges the gap between self-paced studying and real-time collaborative assessment.

The system interfaces with students and administrators through modern desktop and mobile web browsers. It centralizes high-speed AI inference (via Groq API), multi-user test room mechanics, subject-level analytics, and content moderation into a unified, low-latency web platform.

2.2 Product Functionality

1. Student Module

- Authentication & Profile Management: Multi-method registration (Email/Password and Google OAuth), secure session handling, and email-based password recovery.
- Assessment Engine: Exam catalog browsing, dynamic test taking, instant automated scoring, and detailed solution reviews.
- Test Room Mechanics (Collaborative Engine): Test room creation, custom timer/participant configuration, dynamic room code generation, and synchronized test attempts.

- Performance Analytics & Preferences: Subject-wise weak-area analysis, score tracking, and custom question difficulty preferences.
- Community Engagement: Global and peer/friend ranking leaderboards, and a feedback submission pipeline.

2. Admin Module

- System Governance & Moderation : Multi-admin account creation/removal, user access flags, and spam user quarantine. Assessment Engine: Exam catalog browsing, dynamic test taking, instant automated scoring, and detailed solution reviews.
- Content & Catalog Management: Course catalog structure updates, subject mapping, and question bank modification (TestsData editing).
- Feedback & Operations Triage: Platform feedback auditing with status controls (*Bookmark*, *Mark as Spam*, *Mark as Seen*), and global leaderboard monitoring.

2.3 Users and Characteristics

1. Administrators

- Role: System superusers responsible for content accuracy, database integrity, user moderation, and operational oversight.
- Characteristics: Possess strong administrative domain knowledge, technical familiarity with backend content management, and sharp attention to data quality and security flags.

2. Students

- Role: Primary end-users who execute test attempts, host/join live test rooms, monitor learning analytics, and submit system feedback.
- Characteristics: Ranging from diploma students to competitive entrance candidates. Require an intuitive, mobile-responsive UI with zero latency during active test sessions.

Distinguishing Primary Users

1. Administrators (System Integrity Actors): Administrators are the operational backbone. System reliability depends on their ability to curate accurate question banks (TestsData),

flag malicious or spam student accounts, and keep curriculum categories (Subjects/Courses) updated.

2. Students (Primary Value Consumers): Students represent the primary active traffic load on the application. Platform success relies heavily on delivering high engagement through live multiplayer test rooms, real-time ranking updates, rapid question generation, and accurate performance metrics.

2.4 Operating Environment

PrepMaster operates as a web-based, cloud-hosted application accessible across cross-platform client environments (desktops, laptops, tablets, and smartphones).

- Client-Side Environment: Modern web browsers including Google Chrome, Mozilla Firefox, Apple Safari, and Microsoft Edge. Requires standard ECMAScript support for dynamic rendering and client-side state management.
- Network Requirements: Continuous HTTPS internet connectivity for real-time multiplayer test room synchronization, AI inference routing, and live leaderboard polling.
- Server-Side & Hosting Infrastructure: Deployed on Vercel's serverless architecture, backed by cloud database storage and external REST microservices (Groq API and Nodemailer SMTP engine).

2.5 Design and Implementation Constraints

1. Budgetary & API Rate-Limit Constraints

- Description: Infrastructure relies on API tiers (e.g., Groq API rate limits) and serverless cloud execution quotas.
- Impact: Requires efficient prompt caching, request throttling, and optimized backend query structures to prevent cost overruns and service disruptions.

2. Technical & Performance Constraints

- Description: Real-time test room synchronization and instantaneous exam scoring demand minimal network overhead.
- Impact: Database reads/writes must be optimized, and frontend assets minimized (Tailwind CSS v4 engine) to prevent lag during active timed assessments.

3. Data Privacy and Security Constraints

- **Description:** User authentication details, passwords, and feedback logs require strict protection.
- **Impact:** Enforces password hashing, secure JWT/OAuth session handshakes, and input validation to protect against SQL/NoSQL injection and Cross-Site Scripting (XSS).

4. Browser Compatibility Constraints

- **Description:** Varied client rendering engines across desktop and mobile devices.
- **Impact:** Requires responsive CSS design layouts and cross-browser testing to guarantee UI consistency across all screen viewports.

2.6 User Documentation

To ensure smooth onboarding and operational clarity, PrepMaster will deliver:

- **Interactive Onboarding & Tooltips:** Context-sensitive UI hints guiding students through test creation, test room joining via room codes, and analytics interpretation.
- **Admin Operational Manual:** A comprehensive technical guide covering content management (TestsData/Subjects editing), spam user flagging, and feedback moderation pipelines.
- **System API & Schema Guide:** Internal developer documentation detailing database schema entities, sequence call flows, and REST endpoints.

2.7 Assumptions and Dependencies

System Assumptions

- **Client Connectivity:** Users possess stable internet connectivity required for submit actions, real-time leaderboard calculations, and live test room state updates.
- **User Technical Literacy:** Students and administrators possess basic digital literacy for navigating web interfaces and managing account settings.
- **Responsive Viewport Support:** Users will access the platform on standard screen viewports ranging from mobile browsers to high-resolution desktop monitors.

External System Dependencies

- Groq API Cloud: Core dependency for high-speed AI inference, automated question generation, and keyword processing.
- Google OAuth Service: External authentication provider enabling single sign-on (SSO) capabilities.
- SMTP Email Provider: Service dependency (Nodemailer integration) for password recovery links and notification emails.
- Vercel Cloud Platform: Deployment dependency for frontend hosting and serverless API execution.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

- **Design Consistency:** Maintain a unified layout, color palette, typography scale, and component styling across both Student and Admin dashboards using the Tailwind CSS v4 engine.
- **Clarity and Simplicity:** Keep viewports clean and uncluttered by avoiding dense text blocks, utilizing intuitive field labels, and highlighting critical call-to-action buttons like test submission and test room creation.
- **Responsive Layouts:** Guarantee seamless multi-device rendering across mobile viewports, tablet screens, and desktop displays with fluid grid layouts.
- **User-Centered Execution:** Tailor interface workflows specifically to student study patterns and admin moderation tasks to minimize clicks required for common actions.
- **Visual Hierarchy:** Enforce visual hierarchy through contrasting typography sizes, distinct button variants, and status indicators for test progress, room states, and score metrics.
- **Intuitive Navigation:** Provide straightforward sidebar and navbar navigation routes enabling students and admins to switch between dashboards, exam catalogs, test rooms, and feedback panels effortlessly.
- **Robust Error Handling:** Display clear contextual error prompts, form validation messages, and toast notifications when authentication fails or invalid data is entered.
- **Dynamic Action Feedback:** Deliver immediate interface confirmations, progress bars, and loading spinners during asynchronous operations such as test generation, answer saving, and feedback submission.
- **Accessibility Compliance:** Incorporate proper HTML5 semantic elements, highcontrast UI themes, ARIA attributes, and accessible input fields for diverse user accessibility.
- **User Preference Settings:** Allow students to customize question presentation preferences and save platform interface configurations to personal database records.
- **Iterative Prototyping:** Validate UI components and layout flows against architectural wireframes prior to full-stack API integration.

- Continuous UI Refinement: Collect post-deployment usage insights to continuously optimize dashboard layouts and reduce interface friction.

3.1.2 Hardware Interfaces

1. Client Terminal Devices: Support standard desktop workstations, laptops, tablets, and smartphones running modern web browsers without requiring specialized hardware peripherals.
2. Server & Hosting Hardware: Execute on Vercel's distributed serverless infrastructure, backed by scalable cloud database servers and remote API processing units.

3.1.3 Software Interfaces

- Web Application Interfaces: Provide secure web-based frontends tailored to Student and Admin actors for executing tests, managing test rooms, inspecting performance analytics, and moderating content
- External AI Service API: Interface with the Groq API engine via HTTP REST calls to handle high-speed question generation, text parsing, and keyword processing
- Authentication Service API: Integrate with Google OAuth services for single sign-on verification alongside JWT token validation routines
- Email Delivery API: Interface with Node mailer SMTP email services to trigger automated password reset links and transactional notifications
- Database Interface: Connect Express/Node.js backend endpoints to persistent cloud database collections using structured query tools for managing student profiles, test attempts, test rooms, and feedback logs
- External Services Interfaces: Integrate with the Groq API cloud service for high-speed AI inference and utilize Node mailer SMTP integration for dispatching transactional email messages.
- Authentication and Authorization Interfaces: Implement JWT tokens and Google OAuth handshakes to govern role-based access, enforcing strict authorization boundaries between Student and Admin actors.

- **Real-Time Communication Interfaces:** Utilize Socket.io or WebSockets for instantaneous multi-user synchronization within test rooms, dynamic score calculations, and real-time room leaderboards.
- **File and Document Interfaces:** Process file uploads and static assets securely using cloud storage buckets to manage profile images and study export documents.
- **Platform Integration Interfaces:** Expose clean REST API routes to enable potential future integrations with third-party Learning Management Systems and analytics tools.

3.1.4 Communication Interfaces

- **Authentication Page Interface:** Serve login and registration interfaces enabling students and admins to log in via standard credentials or Google OAuth, with password recovery via email.
- **Student Dashboard Interface:** Provide a central dashboard displaying exam selection options, active test room controls, performance overview summaries, and recent feedback status.
- **Admin Dashboard Interface:** Provide a centralized administrative control center for user flagging, admin account creation, question editing, subject categorization, and feedback moderation.
- **Assessment Execution Interface:** Display dynamic test environments with timers, answer selections, question preferences, and instant test submission triggers.
- **Test Room Collaboration Interface:** Render multiplayer test rooms featuring unique room code generation, custom timer/participant settings, and synchronized assessment workflows.
- **Analytics and Solutions Interface:** Present comprehensive performance analysis breakdowns by subject, answer solutions, score reports, and percentage metrics.
- **Leaderboard and Ranking Interface:** Display real-time global rankings and peer leaderboards based on test attempt scores and test room outcomes.
- **Moderation and Feedback Interface:** Enable students to submit platform feedback while providing admins with moderation tools to bookmark, flag as spam, or clear feedback logs.

3.2 Functional Requirements

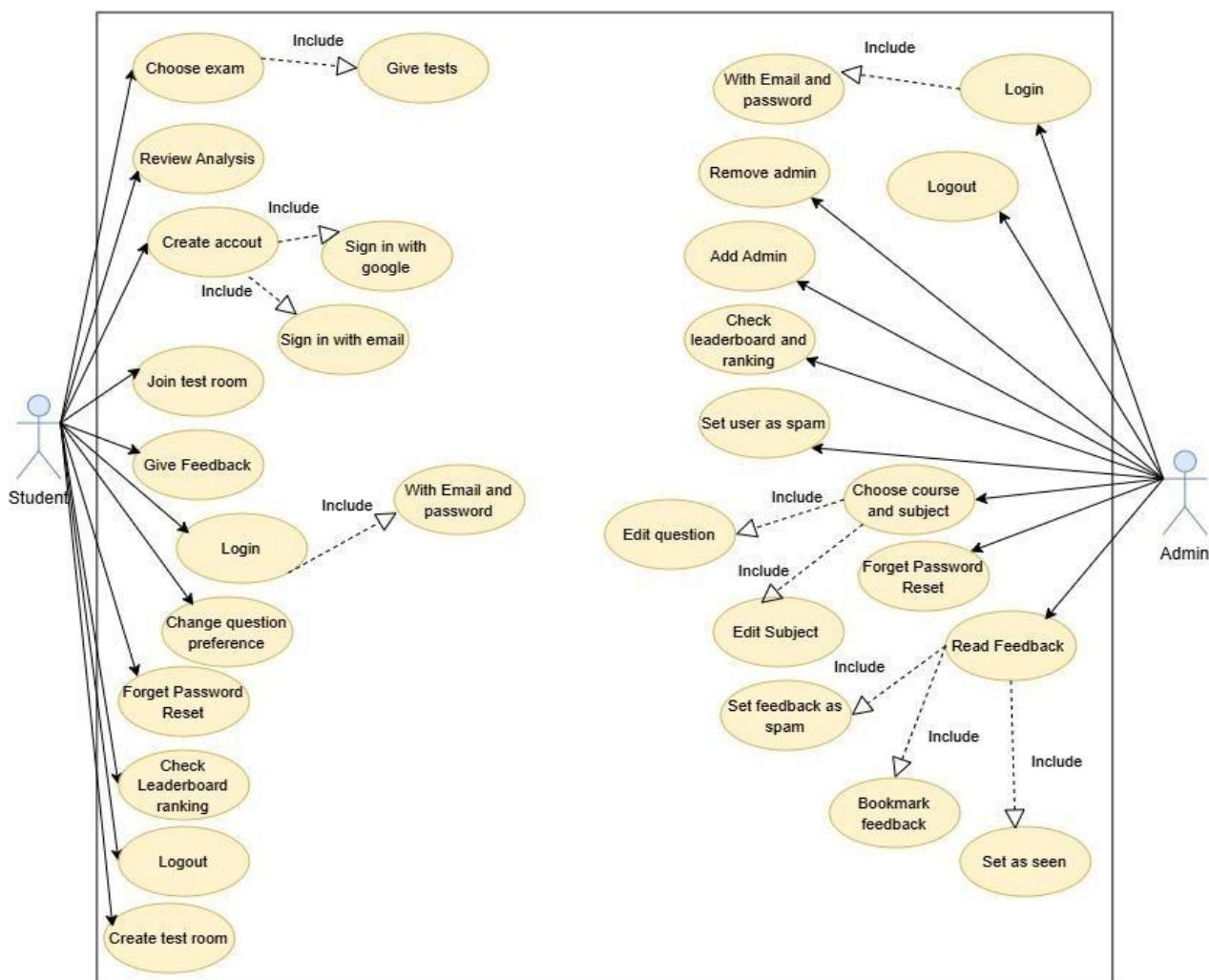
- **User Management:** Process student sign-ups via email or Google OAuth, execute secure logins, handle email password resets, and allow admins to add new admins, remove admins, or flag spam accounts.
- **Assessment and Test Management:** Enable students to browse available exams, customize question preferences, complete timed tests, submit answers, and review score solutions instantly.
- **Test Room Operations:** Allow students to create custom test rooms, configure room timers and participant limits, generate join codes, and participate in collaborative room test sessions.
- **Analytics and Performance Tracking:** Calculate subject-wise performance metrics, track correct and incorrect response rates, and generate overall test attempt reports.
- **Leaderboards and Rankings:** Compute student rankings dynamically across global attempts and friend groups, exposing real-time leaderboard data to both students and admins.
- **Content Management:** Provide admins with complete CRUD permissions over subjects, courses, and test question datasets (TestsData modification).
- **Feedback Management:** Allow students to submit platform feedback, while enabling admins to view, bookmark, set as seen, or mark feedback as spam.

3.3 Behavioral Requirements

3.3.1 Use Case View

- Student Actor Operations: Authenticate, choose exams, complete tests, manage test rooms, review performance analysis, customize preferences, check leaderboards, and submit platform feedback.
- Admin Actor Operations: Authenticate, manage admin credentials, edit courses and subjects, modify test question banks, moderate user flags, monitor leaderboards, and resolve feedback items.

PrepMaster Usecase Diagram



4. Other non functional requirement

4.1 Performance Requirements

- **System Response Time:** Ensure API transactions, test submissions, and dashboard updates complete within a maximum response threshold of less than 2 seconds to deliver a seamless user experience.
- **Concurrent User Execution:** Support a minimum of 500 concurrent active students taking tests and joining live test rooms simultaneously without system degradation.
- **System Scalability:** Architect backend microservices and serverless routes to handle a minimum of 20% growth in simultaneous user traffic during peak exam prep cycles.
- **Data Retrieval Efficiency:** Optimize database indexing for test questions, user attempts, and leaderboards to ensure query response times stay below 1.5 seconds under high load.
- **System Availability Standard:** Maintain a platform uptime benchmark of 99.5% per month, scheduling mandatory system updates during off-peak hours with prior user notification.

4.2 Safety and Security Requirements

- **Data Privacy and Protection:** Secure user profile credentials, password hashes, and feedback records against unauthorized access using industry-standard hashing algorithms.
- **Role-Based Access Control:** Enforce strict authorization boundaries preventing students from accessing administrative functions or modifying question datasets (TestsData).
- **Emergency Data Resilience:** Implement automated database snapshot backups and operational recovery pipelines to preserve test attempts and user progress during unexpected system outages.
- **Encrypted Communications:** Mandate HTTPS encryption for all client-server communications to protect API payloads, auth tokens, and session details in transit.
- **Database Encryption at Rest:** Enforce secure data encryption standards for persistent cloud database storage housing student credentials and system configurations.
- **Audit Logging and Monitoring:** Track critical administrative actions such as user flagging, admin account creation, and question modifications in secure system activity logs.
- **Automated Threat Mitigation:** Protect public endpoints against denial-of-service attempts, cross-site scripting (XSS), and injection attacks through request validation and API rate limiting.
- **Disaster Recovery Protocol:** Conduct regular database recovery drills to guarantee data restoration within target Recovery Time Objectives (RTO).

4.3 Software Quality Attributes

Reliability

- **Comprehensive Error Handling:** Implement standardized backend error middleware and client-side boundary handling to capture, log, and report runtime exceptions without crashing the user session.
- **Informative User Feedback:** Provide intuitive, user-friendly error messages on the frontend during authentication failures, network timeouts, or invalid test room inputs.
- **Fault-Tolerant Architecture:** Design system microservices and edge routes to handle partial service degradation smoothly, preserving ongoing student test states during isolated API failures.
- **Automated Backup Protocols:** Conduct scheduled database backups for user attempts, test room configurations, and system logs to guarantee zero data loss during system recoveries.
- **System Monitoring and Alerts:** Monitor serverless execution health, API response latencies, and database connection metrics to trigger automated administrator alerts upon detecting performance anomalies.

Maintainability

- **Modular Architecture:** Structure the codebase into decoupled frontend components and isolated backend routes (Authentication, Exam Engine, Test Rooms, Leaderboard, Admin Governance) to minimize change ripple effects.
- **Technical Documentation:** Maintain detailed API documentation, clear architectural diagrams, and structured code comments to simplify ongoing maintenance and onboarding.
- **Strict Version Control Workflow:** Manage all application source code, API integration modules, and configuration files via Git repositories using standard branch and pullrequest strategies.
- **Standardized Deployment Pipeline:** Enforce structured change management procedures comprising local testing, automated CI/CD checks on Vercel, and code review approvals prior to production deployments.

Usability

- **Intuitive Interface Design:** Provide clean, distraction-free test environments with minimal visual clutter to allow students to focus purely on exam timing and question solving.
- **Responsive Viewports:** Support fluid layouts across desktop screens, tablets, and mobile devices utilizing modern Tailwind CSS v4 styling strategies.
- **Contextual Guidance:** Offer clear UI feedback including active timers, dynamic question progress trackers, instant error prompts, and confirmation dialogs.

Portability and Compatibility

- **Cross-Browser Support:** Guarantee uniform rendering and full functionality across modern browsers including Chrome, Edge, Safari, Firefox, and mobile web browsers.
- **Platform Independence:** Host backend services on cloud serverless infrastructure, removing client-side hardware dependencies beyond standard web browser access.

Availability and Scalability

- High Uptime Target: Target a 99.5% platform availability standard through Vercel global edge network deployment.
- Elastic Scalability: Leverage serverless compute routing and cloud database indexing to automatically scale during high-traffic exam prep periods.

Testability

- Automated API Validation: Maintain structured unit and integration tests for critical backend routes such as score calculation, room creation, and user authorization handshakes.
- Component Isolation: Structure React frontend components independently to facilitate isolated UI testing and mock data rendering.

Appendix A – Data Dictionary

Table 1.1 Students

ColumnName	ColumnType	Size	Constraint
StudentId	Long	64	Primary Key
FullName	String	100	Not Null
Email	String	320	Not Null
Password	String	256	-
IsSpam	Boolean	1	Not Null

Table 1.2 Admins

ColumnName	ColumnType	Size	Constraint
AdminId	Long	64	Primary Key
FullName	String	100	Not Null
Email	String	320	Not Null
Password	String	256	-
isSuperAdmin	Boolean	1	Not Null

Table 1.3 TestRooms

ColumnName	ColumnType	Size	Constraint
RoomId	Long	64	Primary Key
StudentId	Array	100	Foreign Key
Points	Array	100	Not Null
Difficulty	String	10	Not Null
Correct	Array	100	Not Null
InCorrect	Array	100	Not Null

Table 1.4 TestAttempts

ColumnName	ColumnType	Size	Constraint
TestId	Long	64	Primary Key
StudentId	Long	64	Foreign Key
Points	Int	8	Not Null
Difficulty	String	10	Not Null
Correct	Int	8	Not Null
InCorrect	Int	8	Not Null
UnAttempted	Int	8	Not Null
Attempted	Int	8	Not Null

Table 1.5 Subjects

ColumnName	ColumnType	Size	Constraint
SubjectId	Long	64	Primary Key
SubjectName	String	100	Not Null
CourseId	String	64	Foreign Key

Table 1.6 Courses

ColumnName	ColumnType	Size	Constraint
CourseId	Long	64	Primary Key
Sems	Array	12	Not Null
CourseName	String	100	Not Null

Table 1.7 TestData

ColumnName	ColumnType	Size	Constraint
TestId	Long	64	Primary Key
Questions	Array	50	Not Null

Options	Array	200	Not Null
Answers	Array	50	Not Null

Table 1.8 Feedbacks

ColumnName	ColumnType	Size	Constraint
FeedbackId	Long	64	Primary Key
Description	String	1000	Not Null
Type	String	30	Not Null
isSpam	Boolean	1	Not Null
isBookmark	Boolean	1	Not Null
isSeen	Boolean	1	Not Null
StudentId	Long	64	Not Null

Table 1.9 Leaderboards

ColumnName	ColumnType	Size	Constraint
LeaderboardId	Long	64	Primary Key
StudentId	Array	100	Foreign Key
Points	Array	100	Foreign Key
SubjectId	Long	64	Foreign Key
TotalTests	Array	100	Not Null

Table 1.10 Subscribers

ColumnName	ColumnType	Size	Constraint
SubscriberId	Long	64	Primary Key
StudentId	Long	64	Foreign Key
StartDate	Timestamp	64	Not Null
EndDate	Timestamp	64	Not Null
SubscriptionId	Long	64	Foreign Key

RemainingTokens	Long	64	Not Null
-----------------	------	----	----------

Table 1.11 Subscriptions

ColumnName	ColumnType	Size	Constraint
SubscriptionId	Long	64	Primary Key
Amount	String	20	Not Null
Tokens	Long	64	Not Null
TimeValidity	String	30	Not Null